

Software Requirements Specification (SRS)

Unified Multi-Panel Restaurant, Reservation & Water-Sports Booking Platform

Prepared for:	Wings River Café Management, Laxman Mela Ground, Lucknow
Reference website:	https://wings-river-cafe.pages.dev/
Document version:	1.0
Document type:	SRS + Cost Estimate + Technical Implementation Plan

Document Control & Contents

This document is written in simple, everyday language on purpose — so that the restaurant owner, kitchen staff and any developer or agency can read the same document and understand exactly what is being built, how it works, and what it will cost. It combines three things in one file: the requirements (SRS), the technical plan, and the cost estimate.

1	Introduction & Purpose
2	Project Overview
3	Design & Branding Reference
4	Users & Roles (Who Uses What)
5	Module-wise Feature List (All 5 Panels)
6	Functional Requirements
7	Non-Functional Requirements
8	WhatsApp Notifications — Customer & Manager Side
9	System Architecture
10	Recommended Tech Stack
11	Database — Key Data Entities
12	Screen Count Summary
13	Implementation Plan & Timeline
14	Cost Estimate — Development
15	Cost Estimate — Monthly Running Cost
16	Solo Builder Roadmap
17	Risks & Assumptions
18	Conclusion

1. Introduction & Purpose

Wings River Café (<https://wings-river-cafe.pages.dev/>) is a riverside multicuisine restaurant in Lucknow that also runs a water-sports counter (jet-ski, speedboat, motorboat, kids' rides) and hosts birthday parties and events. Right now the restaurant has a public website. This document describes the plan to build a full **digital operations system** on top of that — one shared backend, with 5 separate simple apps for 5 different kinds of people: customers, waiters, kitchen staff, the manager/reception, and the owner/admin.

The goal is simple: replace paper notepads, shouting across the kitchen, and manual bill calculation with a fast digital system — while keeping every screen so simple that staff can use it without training.

2. Project Overview

The system is split into **5 apps** that all talk to the same central database, so a booking made by a customer instantly appears on the manager's floor map, and an order placed by a waiter instantly appears on the kitchen screen.

- **Customer PWA** — the public website + installable app. Browse menu, reserve a table on a live floor map, order food by scanning a QR code, pay, and track bookings.
- **Waiter PWA** — a big-button app for table status, taking orders, and calling for bills.
- **Kitchen PWA** — one screen showing New → Preparing → Ready → Completed order queue.
- **Reception / Manager PWA** — today's bookings, walk-ins, live floor map, and party bookings.
- **Admin CMS** — the control room: website content, menu, pricing, QR codes, payments, staff, and analytics.

PWA means **Progressive Web App** — in simple words, a website that behaves like a mobile app. Staff can "install" it on their phone home-screen with one tap, it works even on a slightly weak network, and there is nothing to download from the Play Store — which makes it much cheaper and faster to roll out than 5 separate native apps.

3. Design & Branding Reference

The existing site at <https://wings-river-cafe.pages.dev/> already sets the visual identity for the brand, so the new system should continue the same look rather than inventing a new one:

- **Primary colour** — **warm amber/gold (#D97706)**: used as the site's theme colour, on buttons like "Reserve Table" and "Book Party / Event".
- **Mood** — riverside, warm evening-light, fairy lights, sunset tones; dark charcoal backgrounds with amber highlights for a premium waterfront feel.
- **Existing content blocks to reuse** — Hero banner, About, interactive Menu Card (page-flip booklet), Photo Gallery, Water-Sports token/ticket cards, Testimonials, Blog, Location map, Contact form.
- **Existing behaviour to preserve** — WhatsApp click-to-chat button, click-to-call button, Google Maps embed, and an existing **/admin** CMS route that should be replaced by the proper Admin CMS module described in this document.

Recommendation: keep the amber-and-charcoal theme for the Customer PWA (since customers already recognise it), but use a plainer light-background, high-contrast theme for the Waiter, Kitchen and Manager apps — because staff read those screens quickly in a busy, sometimes bright kitchen environment, where a dark decorative theme is harder to scan at a glance.

4. Users & Roles (Who Uses What)

Role	App	Login Needed?	Main Job
Guest / Customer	Customer PWA	Optional (phone/OTP for booking)	Browse, reserve, order, pay, track
Waiter	Waiter PWA	Yes (staff login)	Serve tables, take orders, request bill
Chef / Kitchen staff	Kitchen PWA	Yes (shared kitchen login)	See and update order status
Receptionist / Manager	Manager PWA	Yes (staff login)	Approve bookings, manage floor, walk-ins
Owner / Admin	Admin CMS	Yes (admin login)	Control everything — menu, pricing, staff, reports

All 5 apps share one login system with **role-based access** — meaning the same backend checks who is logging in and shows only the screens that role is allowed to see.

5. Module-wise Feature List (All 5 Panels)

5.1 Customer PWA — 15 screens

Public website and installable app for guests.

- **Home** — hero banner, quick links to Reserve Table, Reserve Party, Menu, Offers, Events, Gallery, Blogs.
- **Menu** — dynamic categories, search, filters, food details, add to cart.
- **Reserve Table** — pick date, time, guest count, and seating cluster.
- **Interactive Floor Map** — BookMyShow-style seat/table picking, shows live availability with a hold timer so two people can't book the same table.
- **Checkout** — booking summary and online payment.
- **Booking Confirmation** — QR ticket + WhatsApp confirmation message.
- **My Bookings** — upcoming, past, cancel, reschedule.
- **Party Booking** — birthday/event canopy booking flow.
- **QR Scan Menu** → **Cart** → **Order Tracking** → **Bill** — a guest sitting at a table scans a QR code, orders food directly, watches the order status live, and views the final bill.
- **Profile, Notifications, Feedback** — account basics, alerts, and reviews.

5.2 Waiter PWA — 6 screens

- **Dashboard** — live table map with colour codes (■ free, ■ eating, ■ needs cleaning, ■ reserved).
- **Active Tables** — one-tap buttons: Check-in, Food Served, Bill Requested, Vacated, Cleaning Done.
- **Orders** — New, Preparing, Ready, Served — so the waiter knows exactly when to carry food out.
- **Call Requests** — instant alerts when a customer asks for water, spoon, tissue, or a waiter from their table.
- **Bills** — generate bill, receive cash, mark paid.
- **Notifications** — all alerts in one place.

5.3 Kitchen PWA — 3 screens

Deliberately the simplest app in the whole system — one big order queue the chef can operate without reading much text: **New Orders** → **Preparing** → **Ready** → **Completed**, moved forward with one tap per order.

5.4 Reception / Manager PWA — 8 screens

- **Dashboard** — today's bookings, walk-ins, current occupancy, upcoming bookings, revenue snapshot.
- **Floor Map** — the whole restaurant with live colour-coded table status.
- **Reservations** — approve, cancel, reschedule, assign table.
- **Walk-in Booking** — quick flow: enter customer → assign table → mark occupied.
- **Party Bookings** — pending, approved, completed.
- **Customers** — booking history, notes, VIP tagging.

5.5 Admin CMS — 12 screens

- **Dashboard** — revenue, occupancy, orders, reservations at a glance.
- **Website CMS** — edit banner, gallery, about, blogs, offers, events, contact, footer, SEO — without touching code.
- **Menu CMS** — categories, food items, combos, images, prices, availability toggle.
- **Reservation Management** — clusters, tables, floor map layout, time slots, maintenance blocking.

- **QR Management** — generate, download, and assign table QR codes.
- **Payments** — transactions, refunds, GST reporting.
- **Analytics** — top-selling food, peak hours, occupancy trends, revenue, customer insights.
- **Staff** — manage waiters, managers, kitchen staff, and their permissions.
- **Settings** — restaurant timing, taxes, service charges, WhatsApp templates, notification templates, payment gateway keys.

6. Functional Requirements

Written as "the system shall..." statements — this is the standard, simple way an SRS describes what the software must actually do.

ID	Requirement
FR-1	The system shall let a guest reserve a table by choosing date, time, guest count and an exact table on a visual floor map.
FR-2	The system shall place a short-duration hold (e.g. 5–10 minutes) on a selected table so two customers cannot double-book it, and release the hold automatically if payment is not completed in time.
FR-3	The system shall generate a QR-coded booking confirmation and send it via WhatsApp.
FR-4	The system shall allow a guest to scan a table's QR code, view the menu, and place an order without waiting for a waiter.
FR-5	The system shall route every new order to the Kitchen PWA in real time, without page refresh.
FR-6	The system shall update a table's status (green/eating/needs-cleaning/reserved) the moment a waiter taps a status button, and reflect it instantly on the Manager's floor map.
FR-7	The system shall let a customer raise a call-request (water / spoon / tissue / waiter) that appears instantly on the assigned waiter's screen.
FR-8	The system shall let the Admin edit menu items, prices, images and availability, and reflect the change on the Customer PWA within seconds.
FR-9	The system shall let the Admin edit website content (banner, gallery, blogs, offers) without any coding.
FR-10	The system shall generate a bill automatically from the order list, apply GST/service charge as configured, and mark it paid once settled.
FR-11	The system shall record every reservation, order and payment against the customer's history for the Manager's "Customers" screen.
FR-12	The system shall restrict every screen to the correct role — a waiter cannot open Admin CMS, a kitchen login cannot open Payments, etc.

7. Non-Functional Requirements

- **Speed** — every screen should feel instant; core actions (reserve, order, status update) in under 2–3 taps, matching the project's own "One Screen = One Primary Action" rule.
- **Real-time** — table status, order status and call-requests must update live across all apps (via WebSockets), not by manual refresh.
- **Reliability** — the table-locking system must never allow two bookings to clash for the same slot.
- **Usability** — Waiter and Kitchen apps must be usable with zero training, using large buttons and colour, not long text.
- **Installability** — every app must be installable to a phone home-screen (PWA) and usable in low-signal areas near the riverfront.
- **Security** — role-based login, encrypted passwords, and secure payment handling (no card data stored on our own server).
- **Scalability** — the same backend must comfortably handle festival/weekend rush — many simultaneous bookings and orders.
- **Availability** — target uptime 99.9%, since the system runs the live restaurant floor, not just a website.

8. WhatsApp Notifications — Customer & Manager Side

WhatsApp is the main way this system talks to people outside the app — because customers already expect a WhatsApp confirmation (as seen on the current site's "Book Table / Party" button), and staff need instant alerts even when they haven't opened the Manager PWA. Every message below is sent automatically by the backend the moment the matching event happens — no one has to type or send anything by hand.

8.1 Customer-side WhatsApp Notifications

Trigger (What Just Happened)	Message Sent to Customer
Table reservation confirmed	Booking confirmation with date, time, table, and a QR ticket to show at entry
Party / event booking approved	Confirmation with event date, canopy/venue, and any deposit/payment link
Booking reminder	A short reminder sent a few hours before the reservation time
Table hold about to expire	A nudge if a selected table is on hold but payment/checkout isn't finished yet
Order placed via QR menu	Order summary — items, quantity, estimated time
Order status changes to "Ready"	"Your food is on its way to your table"
Bill generated	Itemised bill with GST/service charge, and payment link if paying online
Payment received	Payment receipt / thank-you message
Booking cancelled or rescheduled	Updated confirmation reflecting the new status
Feedback request	Sent after the visit, with a link to leave a review

8.2 Manager / Staff-side WhatsApp Notifications

Trigger (What Just Happened)	Message Sent to Manager / Staff
New table reservation comes in	Alert with customer name, date, time, guest count — for quick approval
New party / event booking request	Alert with event details, so the manager can approve or follow up
Booking cancelled by customer	Alert so the table can be released and re-offered
Walk-in seated by reception	Confirmation logged, visible in the Manager dashboard summary
Unresolved call request	Escalation alert to the manager if a waiter hasn't cleared a water/spoon/bill request within a set time
Bill generated / payment received	Confirms the amount and payment method, for reconciliation
End-of-day summary	Total bookings, occupancy, orders and revenue for the day, sent once at closing time

8.3 How It Works (In Simple Terms)

- Every message uses a **pre-approved WhatsApp template** — Meta (WhatsApp's parent company) reviews and approves the exact wording once, and after that the system can send it automatically as many times as needed.

- The **Admin CMS** → **Settings** → **WhatsApp Templates** screen (already part of the Admin module) is where the restaurant owner can view and update the wording of these templates without touching code.
- For customers, only people who make a booking or place an order receive messages — this is opt-in by nature of giving their phone number at checkout, not a marketing blast.
- As a low-cost starting point before full WhatsApp Cloud API approval comes through, the same confirmations can be sent as a **click-to-chat (wa.me) link** pre-filled with the booking details, exactly like the current site's "Book Table / Party" WhatsApp button — this needs no approval and can go live on day one.

Setup note: getting full automated WhatsApp Cloud API access requires a one-time Meta Business verification, which typically takes a few days — it's listed under Risks & Assumptions later in this document, and should be started early in the build.

9. System Architecture

In simple words: one shared "engine" (backend) runs everything, and the 4 staff apps + 1 customer app are just different "screens" plugged into that same engine — so data never gets out of sync between them.

Layer	What it does	Technology
1. Front-end apps	The 5 installable PWAs the users actually see and tap	React + PWA, hosted on Cloudflare Pages
2. API layer	Receives every request (book a table, place an order, update status) and applies business rules	Hono framework on Cloudflare Workers
3. Real-time layer	Keeps table-hold locking accurate and pushes live updates instantly to every open screen	Cloudflare Durable Objects + WebSockets
4. Database	Stores every booking, order, menu item, bill and user	Cloudflare D1 (SQL database)
5. Notifications	Sends booking confirmations and alerts	WhatsApp Cloud API / click-to-chat
6. Payments	Collects online payments for bookings and food	Razorpay or similar Indian payment gateway

Why this shape works well for a single restaurant: all 5 apps calling the same API means a single source of truth — a table can never show "free" on one screen and "occupied" on another. It also means new features (say, a 6th "Valet PWA" later) can be added without rebuilding anything that already exists.

Simple Request Flow (example: table booking)

- **Step 1.** Guest opens Customer PWA (website or installed app)
- **Step 2.** Guest reserves a table or scans a QR code to order
- **Step 3.** Request goes to the Hono API on Cloudflare Workers
- **Step 4.** Durable Objects lock the table / update live order state
- **Step 5.** Cloudflare D1 database saves the booking / order permanently
- **Step 6.** Waiter, Kitchen and Manager PWAs instantly receive the update over WebSocket

10. Recommended Tech Stack

Part	Recommended Technology	Why
Front-end (all 5 apps)	React + Vite, Tailwind CSS, PWA plugin	One shared codebase style, fast to build, installable, matches existing site's tech feel
Real-time floor map / seat picking	Cloudflare Durable Objects + WebSocket	Purpose-built for "only one person can hold this table at a time" logic
Backend / API	Hono (lightweight web framework) on Cloudflare Workers	Very fast, runs close to the user, low hosting cost, no server to manage
Database	Cloudflare D1 (SQLite-based SQL database)	Simple relational data (bookings, orders, menu) fits SQL well; integrates natively with Workers
File / image storage	Cloudflare R2 or Cloudflare Images	Menu photos, gallery, QR codes

Part	Recommended Technology	Why
Authentication	JWT-based auth with role field (customer / waiter / kitchen / manager / admin)	One login system, different screens per role
Payments	Razorpay (UPI, cards, net-banking)	Most common, trusted gateway for Indian restaurants
Notifications	WhatsApp Cloud API (or wa.me links to start)	Customers already expect WhatsApp confirmations, as seen on the current site
QR generation	qrcode.react / server-side QR library	For table QR menus and booking tickets
Hosting	Cloudflare Pages (front-end) + Cloudflare Workers (backend)	One vendor, same account as the current site, very low running cost

Note: this stack matches the architecture diagram already sketched in the project brief (Cloudflare Pages → Hono → Durable Objects → D1), so it is not a new suggestion — this document simply confirms it is the right, cost-efficient choice for a single-location restaurant like this.

11. Database — Key Data Entities

In simple words, these are the main "lists" the system needs to remember:

Entity	Stores
Users	All logins — customers, waiters, kitchen, managers, admin — with role
Tables / Clusters	Every physical table, its cluster/zone, seating capacity, and floor-map position
Reservations	Date, time, guest count, table, status (pending/approved/cancelled), hold-timer expiry
Party Bookings	Event date, canopy/venue, guest count, custom menu notes, approval status
Menu Items	Category, name, price, image, availability, combo grouping
Orders & Order Items	Which table/customer, which items, quantity, status (new/preparing/ready/served)
Bills / Payments	Amount, GST, service charge, payment method, paid/unpaid status
QR Codes	Which table or menu a QR code links to
Call Requests	Table, request type (water/spoon/tissue/waiter), time raised, resolved status
Website Content	Banners, gallery images, blogs, offers, events, footer, SEO fields
Customers	Contact info, booking history, notes, VIP flag

12. Screen Count Summary

Module	Screens
Customer PWA	15
Waiter PWA	6
Kitchen PWA	3
Reception / Manager PWA	8
Admin CMS	12
Total	44+

The golden rule for every one of these screens, as set out in the original brief: **"One Screen = One Primary Action."** Customers reserve/order in 2–3 taps, waiters update status in one tap, kitchen only sees the order queue, and the manager sees the live floor map plus bookings — nothing more.

13. Implementation Plan & Timeline

The phases below assume a small **team** (designer + developer working in parallel). Since you are building this **solo**, see the adjusted phase table and roadmap notes right after it — the module order changes slightly to suit one person.

Phase	What happens	Duration (Team)
Phase 1 — Foundation	Backend setup (Hono + D1), shared login system, database design, design system (colours/fonts) carried over from the current site	1–2 weeks
Phase 2 — Customer PWA	Home, Menu, Reserve Table + live floor map, Checkout, Booking confirmation, My Bookings, QR ordering	3–4 weeks
Phase 3 — Waiter + Kitchen PWA	Table dashboard, order flow, call requests, billing, kitchen queue	1.5–2 weeks
Phase 4 — Manager PWA	Dashboard, floor map, reservations, walk-ins, party bookings, customer history	1.5–2 weeks
Phase 5 — Admin CMS	Website CMS, Menu CMS, reservation & QR management, payments, analytics, staff, settings	2–3 weeks
Phase 6 — Testing & Launch	End-to-end testing on real tables, staff training (should need very little, by design), go-live	1 week

Total for a small team: roughly **10–14 weeks (about 2.5–3.5 months)**.

14. Cost Estimate — Development (One-Time)

These are typical India-market rates for a project of this size (44+ screens, 5 apps, real-time features, payment integration). Actual quotes will vary by city and by the freelancer/agency you choose — treat this as a planning range, not a fixed quote.

Work Item	Description	Estimated Cost (₹)
UI/UX Design	Screen designs for all 44+ screens, carrying forward the amber-riverside theme	20,000 – 40,000
Customer PWA (15 screens)	Website + booking + floor map + QR ordering + payments	45,000 – 90,000
Waiter PWA (6 screens)	Table status, orders, call requests, billing	15,000 – 30,000
Kitchen PWA (3 screens)	Order queue screen	8,000 – 15,000
Manager PWA (8 screens)	Dashboard, floor map, reservations, walk-ins	20,000 – 40,000
Admin CMS (12 screens)	Website CMS, menu CMS, payments, analytics, staff, settings	35,000 – 70,000
Backend, database & real-time engine	Shared API, login system, Durable Objects table-locking, WebSocket live updates	50,000 – 1,00,000
Payment gateway & WhatsApp integration	Razorpay setup, WhatsApp Cloud API / template messages	10,000 – 20,000
QR code system	Table QR generation, download, assignment	5,000 – 10,000
Testing & deployment	End-to-end testing, staff walkthrough, go-live support	10,000 – 20,000

Total (one-time development)	≈ ■2,18,000 – ■4,35,000
-------------------------------------	--------------------------------

*These figures are what this project would cost to **outsource**. Since you're building it yourself, treat this table as the value of the work rather than money you'll spend — your real cost is time, not cash (see the Solo Builder Roadmap for a realistic time estimate). It's still useful for deciding whether to bring in paid help for one specific piece later — e.g. hiring a designer for a week, or a freelancer just for the real-time locking engine — without handing over the whole project.*

15. Cost Estimate — Monthly Running Cost

Item	Typical Monthly Cost (■)	Notes
Cloudflare Pages hosting	Free	Free tier is generous for a single-restaurant site
Cloudflare Workers Paid plan	≈ 400 (US\$5)	Needed to unlock full D1 + Durable Objects usage at this scale
Domain name	≈ 70/month (≈ 800–1,200/year)	Renewed yearly
WhatsApp Cloud API messages	≈ 500 – 2,000	Depends on number of booking confirmations sent per month; free via wa.me click-to-chat as a starting alternative
Razorpay payment gateway	No fixed fee	≈ 2% transaction fee only on successful online payments
Ongoing support / minor updates	≈ 3,000 – 8,000	Optional monthly retainer for bug-fixes and small changes

Rough total: about ■1,000 – 2,500/month in pure hosting/infrastructure, plus an optional support retainer. This is low because the chosen stack (Cloudflare) has no traditional server to rent — you only pay for actual usage.

16. Solo Builder Roadmap (Building Alone)

Building this alone is realistic — but the order you build things in, and one structural decision, make a big difference to how much of it you actually have to write.

16.1 The One Decision That Saves You the Most Time

Don't build Waiter, Kitchen, and Manager as 3 separate codebases. Build them as **one "Staff PWA" codebase with role-based views** — same login screen, same shared components (buttons, table cards, order cards), and the backend simply decides which screens to show based on who logged in. Staff still install it and experience it as their own dedicated app; you just don't triple your front-end work maintaining three near-identical apps. This single change is usually the difference between a solo project that finishes and one that stalls.

16.2 Recommended Build Order

Step	Build This	Why This Order
1	Backend, database, and login/role system	Everything else depends on this; get bookings, orders, users and tables modelled correctly before writing any screen.
2	Real-time engine — table hold-locking + WebSocket live updates	The riskiest, most technical part. Prove it works with a bare-bones test screen before building real UI on top of it.
3	Customer PWA — Reserve Table + Floor Map + Checkout first, QR ordering next, rest of the content pages last	This is the part that actually earns money (bookings, orders) — build it before anything internal-facing.
4	Staff PWA — Kitchen view first, then Waiter view, then Manager view	Kitchen is simplest (good confidence win), Waiter reuses the same order/table data, Manager reuses the same floor map you already built for customers.

Step	Build This	Why This Order
5	Admin CMS — Menu CMS and QR Management first; Website CMS, Analytics, Settings last	Until this exists, edit menu items and prices directly in the database yourself — normal for a solo v1.
6	WhatsApp notifications	Start with free wa.me click-to-chat links (no approval needed); upgrade to full Cloud API templates once Meta verification is done, in parallel with other work.
7	Testing, staff walkthrough, go-live	Test on real tables with real staff before a full launch.

16.3 Realistic Solo Timeline

Pace	Estimated Time
Full-time (building this as your main job)	≈ 4 – 6 months
Part-time (evenings & weekends alongside other work)	≈ 7 – 10 months

This is longer than the 2.5–3.5 month team estimate mainly because one person is doing design, backend, front-end, and testing in sequence rather than in parallel — not because the work itself is different.

16.4 Practical Tips for Building Solo

- **Ship Phase 3 (Customer PWA) and start using it** before building the rest — real bookings coming in will surface bugs faster than any amount of solo testing, and it starts paying for itself immediately.
- **Reuse UI components aggressively** — a "table card", an "order card", and a "status button" appear in almost every screen across all apps; build each one once, well, and reuse it everywhere.
- **Keep the Admin CMS manual at first** — a simple internal screen (or even direct database edits) for updating menu prices is fine for months; don't let a polished CMS block your launch.
- **Use AI coding assistants (like Claude Code) for repetitive screens** — many of the 44+ screens follow the same pattern (list → detail → action), which is exactly the kind of repetitive front-end work that speeds up a lot with AI-assisted coding, freeing your own time for the real-time engine and business logic.
- **Set a hard "good enough for v1" bar** — features like Analytics, VIP notes, and SEO fields in the Admin CMS can wait until after launch; the floor map, ordering, and billing cannot.

17. Risks & Assumptions

- **Assumption:** the restaurant has reasonably stable Wi-Fi/4G near the riverfront seating for real-time features to work well; a fallback "offline queue" for orders is recommended if signal is patchy in some outdoor sections.
- **Assumption:** staff have basic smartphones (Android is fine — PWAs work in any modern mobile browser).
- **Risk:** real-time table-locking (Durable Objects) is the most technically complex piece — it should be built and tested first, since every other booking feature depends on it working correctly.
- **Risk:** WhatsApp Cloud API requires Meta Business verification, which can take a few days — start this early so it doesn't delay launch.
- **Risk:** during festival rush (peak evenings/weekends), table QR ordering could spike; the Cloudflare Workers Paid plan comfortably handles this, but it's worth load-testing once close to launch.

18. Conclusion

This system takes the existing Wings River Café brand and public website and extends it into a complete, connected restaurant operating system — one login system, one database, five screens tailored to five different jobs. Built on Cloudflare's Pages + Workers + D1 + Durable Objects stack, it keeps running costs low (roughly ₹1,000–2,500/month) while giving the restaurant real-time control over tables, orders, bookings, and the water-sports counter — all for a one-time development investment in the ≈ ₹2–4.5 lakh range, depending on who builds it.

— End of Document —